

Laboratório 2 - MC714

Disciplina: MC714 - Sistemas Distribuídos

Professor: Luiz Fernando Bittencourt

Integrantes:

- Anita Ramos Almeida – RA: 173273
- Wellington Tavares da Silva – RA: 195549

Link do repositório: [lab_2_github](#)

Link do vídeo: [lab_2_vídeo](#)

1. Introdução

Este relatório apresenta o desenvolvimento do segundo trabalho da disciplina MC714 - Sistemas Distribuídos, cujo objetivo foi implementar três algoritmos clássicos de coordenação distribuída usando troca de mensagens entre processos. Os algoritmos foram: relógio lógico de Lamport, exclusão mútua distribuída e eleição de líder.

O projeto implementado consiste em um conjunto de programas em Python que simulam nós distribuídos executando em máquinas distintas da Google Cloud. Cada nó roda três algoritmos em portas diferentes: Lamport, exclusão mútua e Bully. A comunicação entre os nós é feita por sockets TCP, com mensagens em formato JSON. Também foi desenvolvido um dashboard simples, separado dos algoritmos, para visualizar o estado dos nós durante a execução.

A escolha por sockets TCP foi feita porque eles permitem representar uma comunicação real entre processos distribuídos, sem depender de arquivos compartilhados ou de uma simulação local. Com isso, o projeto fica mais próximo do comportamento esperado em sistemas distribuídos reais, nos quais cada nó possui seu próprio estado local e precisa trocar mensagens com os demais para coordenar decisões.

2. Problema

Em sistemas distribuídos, diferentes processos executam de forma independente e muitas vezes em máquinas diferentes. Como não existe uma memória global compartilhada nem um relógio físico perfeitamente sincronizado entre todos os nós, surgem problemas importantes de coordenação. Entre eles, se destacam a ordenação de eventos (determinação da ordem lógica entre eventos que ocorrem em nós diferentes), a exclusão mútua (garantia de que um processo acesse a região crítica por vez) e eleição de líder (escolha de um coordenador quando o sistema inicia ou quando o líder atual falha).

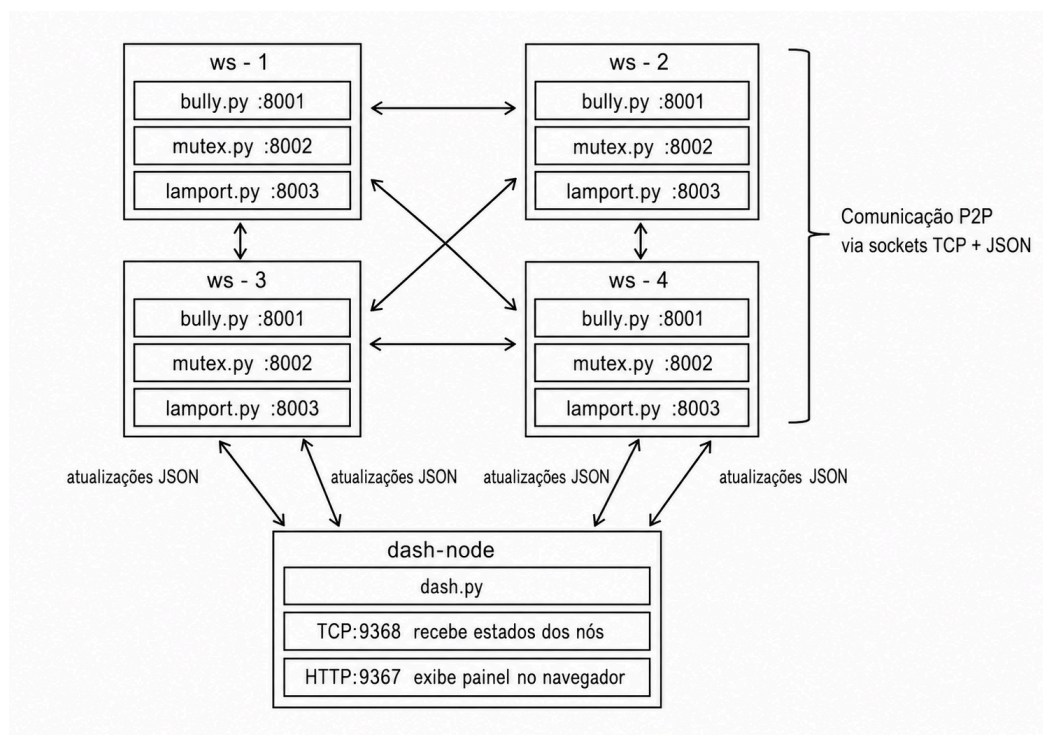
Esses problemas são fundamentais em sistemas distribuídos, porque aparecem em diversas situações como acesso a banco de dados, coordenação de servidores, replicação de dados, controle de recursos compartilhados e tolerância a falhas.

No projeto, esses três problemas foram tratados por meio da implementação de três algoritmos: o Relógio Lógico de Lamport (para ordenação lógica de eventos), o Algoritmo de Ricart-Agrawala (para exclusão mútua distribuída) e o Algoritmo Bully (para eleição de líder).

3. Visão geral da solução

A solução foi organizada em módulos independentes, mas que compartilham uma camada comum de comunicação. O repositório contém os arquivos principais `base.py`, `lamport.py`, `mutex.py`, `bully.py`, `dash.py` e scripts de apoio para criação, implantação e encerramento das máquinas na Google Cloud. O código `base.py` abstrai a comunicação P2P com sockets TCP e threads, expondo funções para inicializar um nó e enviar mensagens, além do código `dash.py`, que atua como painel de acompanhamento separado, recebendo atualizações dos nós.

A arquitetura geral pode ser representada da seguinte forma:



Foram utilizadas quatro máquinas de trabalho, chamadas ws-1, ws-2, ws-3 e ws-4, além de uma máquina separada chamada dash-node, responsável apenas pelo dashboard. Essa separação é importante, porque o dashboard não participa das decisões dos algoritmos, ou seja, ele só recebe mensagens de estado para facilitar a demonstração e análise.

4. Ambiente de execução

O projeto foi executado em máquinas virtuais na Google Cloud. O script `setup.sh` configura a zona “us-central1-a”, cria as instâncias `ws-1`, `ws-2`, `ws-3`, `ws-4` e `dash-node`, usa o tipo de máquina `e2-medium`, instala Python 3, pip e Git, e cria uma regra de firewall liberando tráfego TCP no intervalo de portas necessário para os algoritmos e para o dashboard (8000-9368).

A configuração principal do ambiente foi:

- Linguagem de programação: Python 3
- Comunicação: Sockets TCP
- Formato das mensagens: JSON
- Infraestrutura: Google Cloud
- Máquinas de “trabalho”: `ws-1`, `ws-2`, `ws-3`, `ws-4`
- Máquinas de “visualização”: `dash-node`
- Porta do Bully: 8001
- Porta da exclusão mútua: 8002
- Porta do Lamport: 8003
- Porta TCP do dashboard: 9368
- Porta web do dashboard: 9367

Como Python é uma “linguagem interpretada”, não há uma etapa de compilação, então a execução depende apenas de Python 3 e das bibliotecas utilizadas, que são bibliotecas padrão da linguagem, como `socket`, `threading`, `json`, `time`, `csv`, `random` e `http.server`.

5. Camada de comunicação

A camada de comunicação foi implementada no arquivo `base.py`. Esse módulo concentra a lógica comum de envio e recebimento de mensagens entre os nós. Ele usa sockets TCP, codificação JSON e threads para que cada nó consiga escutar mensagens em segundo plano enquanto continua executando seu algoritmo principal.

O módulo tem duas funções principais:

- `init(ip, porta, callback)`: inicializa o servidor TCP do nó em uma thread separada e define a função que será chamada quando uma mensagem chegar
- `snd(ip_destino, porta_destino, mensagem)`: abre uma conexão TCP com o nó de destino, serializa a mensagem em JSON e envia os dados

Na implementação, o servidor cria um socket TCP, usa `SO_REUSEADDR`, faz o bind no IP e porta configurados, entra em modo de escuta e a cada conexão recebida decodifica o JSON e repassa a mensagem para a função de callback do algoritmo. A função de envio cria o socket, conecta ao destino, serializa a mensagem usando `json.dumps`, envia com `sendall` e

fecha a conexão no final. Essa separação deixou os algoritmos mais simples, porque cada um deles foca na sua regra de coordenação, sem repetir toda a lógica de socket.

6. Dashboard de acompanhamento

Além dos algoritmos distribuídos, foi implementado um dashboard para acompanhar o estado dos nós. O dashboard não interfere no funcionamento dos algoritmos, ele apenas recebe atualizações de estado enviadas pelos nós.

O arquivo dash.py cria dois serviços:

1. Um servidor TCP na porta 9368, usado pelos nós para enviar atualizações em JSON
2. Um servidor HTTP na porta 9367, usado para visualizar os estados no navegador

A página web do dashboard atualiza periodicamente os dados consultando a rota /d, que retorna o estado atual dos nós em JSON. A interface mostra o identificador, status online/offline, valor do relógio lógico, líder atual e estado da exclusão mútua para cada nó. O servidor TCP recebe as mensagens dos nós, atualiza a estrutura de estado compartilhada e registra um timestamp para indicar se o nó ainda está ativo. Isso facilitou bastante a demonstração no vídeo, pois permitiu observar em uma única tela o comportamento dos três algoritmos juntos.

7. Algoritmo 1: Relógio lógico de Lamport

7.1 Algoritmo

O relógio lógico de Lamport resolve o problema de ordenar eventos em sistemas distribuídos sem depender de relógios físicos sincronizados, garantindo uma ordenação lógica que mantém a relação de causalidade entre eventos em sistemas distribuídos. A ideia central é que cada processo mantém um contador lógico local, que é incrementado quando ocorre um evento interno ou envio de mensagem. Quando um processo recebe uma mensagem, ele atualiza o seu relógio para um valor maior do que o máximo entre seu relógio local e o relógio recebido.

A regra principal utilizada é:

- Envio de mensagem: $C_i = C_i + 1$
- Recebimento de mensagem com timestamp C_m : $C_i = \max(C_i, C_m) + 1$

7.2 Implementação

No arquivo `lamport.py`, cada nó possui uma variável global `c`, que representa o relógio lógico local. Também foi utilizado um `threading.Lock` para proteger o acesso concorrente ao relógio, já que a thread principal e a thread de recebimento de mensagens podem acessar esse valor ao mesmo tempo.

A função `evt()` incrementa o relógio local quando ocorre um envio. A função `rcv(m)` atualiza o relógio ao receber uma mensagem, usando a regra $\max(c, \text{relógio_recebido}) + 1$. O código também registra no log se o evento foi de envio ou recebimento.

A cada intervalo definido no arquivo `nodes.csv`, o nó escolhe aleatoriamente um dos pares conhecidos e envia uma mensagem contendo seu identificador e o valor atual do relógio lógico. O código lê `nodes.csv`, identifica o IP e o heartbeat do nó atual e adiciona os demais nós à lista de pares.

7.3 Mensagens utilizadas

A mensagem do Lamport possui formato simples, contendo o “id”, que representa o identificador do nó emissor, e o “c”, que representa o valor do relógio lógico do emissor no momento do envio.

7.4 Resultado

Nos testes, quando um nó recebe uma mensagem com um relógio maior que o seu valor local, ele atualiza seu contador para manter a ordem causal. O dashboard mostra a evolução do campo CLK, enquanto as últimas mensagens de envio e recebimento podem ser visualizadas diretamente no log.

Esse comportamento confirma que a implementação respeita a regra essencial do relógio lógico de Lamport de que se um evento causa outro por meio de comunicação, então o timestamp lógico do primeiro evento será menor que o timestamp lógico do segundo.

8. Algoritmo 2: Exclusão mútua distribuída

8.1 Problema

O problema de exclusão mútua consiste em garantir que apenas um processo por vez acesse uma região crítica. Em sistemas distribuídos, os processos não compartilham memória e precisam coordenar o acesso por troca de mensagens de maneira que dois nós não entrem em uma região crítica ao mesmo tempo.

8.2 Algoritmo

O algoritmo implementado é baseado no Ricart-Agrawala. Nesse algoritmo, quando um processo deseja entrar na região crítica, ele envia uma requisição para os demais

processos. Cada processo responde imediatamente ou adia a resposta, dependendo de sua própria situação. O processo que solicitou só entra na região crítica após receber permissão dos demais processos ativos.

8.3 Estados do nó

Na implementação, o estado local do nó é representado pela variável *s*, tal que:

- *s* = 0 representa um nó livre, ou seja, que não quer a região crítica
- *s* = 1 representa um nó aguardando permissão para acessar a região crítica
- *s* = 2 representa um nó dentro da região crítica

Além disso, a implementação usa:

- *c*: relógio lógico local
- *r_c*: timestamp do pedido atual
- *rep*: quantidade de respostas recebidas
- *exp*: quantidade de respostas esperadas
- *q*: fila de respostas adiadas
- *lck*: lock para proteger variáveis compartilhadas

8.4 Funcionamento

Quando um nó decide solicitar a região crítica, ele executa a função *b_req()*. Essa função muda o estado para aguardando, incrementa o relógio lógico, registra o timestamp do pedido e envia mensagens REQ aos demais nós ativos. Antes de enviar a requisição, a implementação tenta abrir uma conexão com o par, usando timeout, para contar apenas os nós alcançáveis com as respostas esperadas.

Quando um nó recebe uma mensagem REQ, ele decide se responde imediatamente com REP ou se adia a resposta. A resposta é adiada quando o nó já está na região crítica ou quando também está aguardando a região crítica e possui prioridade maior. A prioridade é decidida pelo par (timestamp, id), o que evita empate entre requisições.

Quando um nó recebe mensagens REP suficientes, ele entra na região crítica, muda seu estado para *NA_REGIAO_CRITICA* e atualiza o dashboard. Após permanecer 5 segundos nessa região, ele libera o recurso, muda o estado para *LIBERADO* e envia as respostas que haviam ficado pendentes na fila.

8.5 Mensagens utilizadas

As mensagens de exclusão mútua têm o formato contendo o “id” que representa o identificador do nó emissor, o “t” que representa o tipo da mensagem (REQ ou REP) e o “c” que representa o valor do relógio lógico associado à mensagem.

8.6 Resultado

Durante os testes, o dashboard mostrou os nós alternando entre os estados AGUARDANDO, NA_REGIAO_CRITICA e LIBERADO. A região crítica é ocupada por um nó por vez, enquanto os outros aguardam ou permanecem liberados.

9. Algoritmo 3: Eleição de líder com Bully

9.1 Problema

Em sistemas distribuídos, muitas vezes é necessário escolher um processo coordenador responsável por centralizar decisões, organizar tarefas ou representar o grupo. Porém, se o líder falha, os demais processos precisam detectar a falha e eleger um novo líder.

O algoritmo Bully resolve esse problema usando identificadores de prioridade. A regra principal é que o processo com maior identificador entre os processos ativos deve se tornar o líder.

9.2 Funcionamento do Bully

O funcionamento conceitual é:

1. Um nó percebe que não há líder ou que o líder parou de responder
2. Ele envia mensagens de eleição para os nós com identificador maior
3. Se algum nó maior responde, esse nó maior assume a responsabilidade de continuar a eleição
4. Se ninguém maior responde, o próprio nó se declara líder
5. O novo líder anuncia sua decisão para os nós de menor identificador

9.3 Implementação

No arquivo bully.py, cada nó mantém as variáveis:

- ldr: líder conhecido
- elc: indica se o nó está em eleição
- ok_r: indica se recebeu resposta de um nó de maior prioridade
- last_hbt: instante do último heartbeat recebido do líder
- pr: lista de pares conhecidos

A implementação usa quatro tipos principais de mensagem:

- HBT: heartbeat enviado pelo líder
- ELC: mensagem de início de eleição
- OK: resposta de um nó de maior prioridade
- COR: mensagem anunciando o coordenador/líder

Quando o nó recebe uma mensagem ELC, ele responde com OK e, se ainda não estiver em eleição, inicia sua própria eleição. Quando recebe COR, atualiza o líder conhecido.

A função `strt()` implementa o início da eleição. Ela procura nós com identificador maior e, se não tiver nenhum, o próprio nó se declara líder e envia COR aos nós menores. Se tiver nós maiores, envia ELC para eles e espera por resposta. Caso não receba OK, ele se declara líder.

A função `monitor_lider()` envia heartbeats quando o nó é líder e, quando o nó não é líder, verifica se o líder atual parou de responder. Se o tempo desde o último heartbeat ultrapassar $hb * 3$, o nó considera que o líder caiu e inicia uma nova eleição.

9.4 Resultado

No cenário com quatro nós ativos, o nó de maior identificador, ws-4, tende a ser eleito líder. Quando o processo do Bully no nó líder é encerrado, os demais nós param de receber heartbeats. Após o timeout, eles iniciam uma nova eleição e o maior identificador ainda ativo passa a ser o novo líder.

O projeto também inclui scripts específicos para derrubar e restaurar apenas o processo `bully.py` em uma máquina. O script `bully_stop.sh` encerra o processo do Bully em uma instância específica sem parar os demais algoritmos, enquanto `bully_start.sh` reinicia o processo em uma máquina e nó escolhidos.

10. Scripts de implantação e execução

O projeto contém scripts para facilitar a criação e execução do ambiente.

10.1 Criação da infraestrutura

O script `setup.sh` cria as máquinas virtuais e a regra de firewall. Ele cria as instâncias ws-1, ws-2, ws-3, ws-4 e dash-node, instala dependências básicas e libera o intervalo de portas TCP usado pelos algoritmos e pelo dashboard.

10.2 Implantação do dashboard

O script `deploy_dash.sh` acessa a máquina dash-node, instala Git e Python 3, clona ou atualiza o repositório e inicia o processo `dash.py`.

10.3 Implantação dos nós

O script `deploy_ws.sh` recebe o identificador do nó como argumento, descobre os IPs internos das máquinas, cria o arquivo `nodes.csv` com os quatro nós e seus intervalos, copia esse arquivo para a máquina correspondente e inicia os três programas do projeto.

No `nodes.csv`, os nós são configurados com identificadores de 1 a 4 e intervalos de 2, 3, 5 e 7 segundos. O script também inicia `bully.py` na porta 8001, `mutex.py` na porta 8002 e `lamport.py` na porta 8003.

10.4 Encerramento do ambiente

Ao final dos testes, o script `shutdown.sh` remove as instâncias e a regra de firewall criada, evitando o consumo desnecessário de recursos na Google Cloud.

11. Testes realizados

11.1 Teste de inicialização

O primeiro teste foi verificar se as máquinas eram criadas corretamente, se o código era clonado do GitHub e se os três algoritmos iniciavam em cada nó. O sucesso desse teste foi observado pela presença dos processos em execução e pela chegada de atualizações no dashboard. O dashboard passou a exibir os nós ativos, as colunas de relógio lógico, líder e exclusão mútua começaram a ser preenchidas e os logs dos processos mostraram mensagens enviadas e recebidas.

11.2 Teste do relógio lógico de Lamport

Neste teste, os nós enviaram mensagens periodicamente para pares escolhidos aleatoriamente. O objetivo foi verificar se o relógio lógico local aumentava corretamente a cada envio e recebimento. Os relógios aumentaram ao longo do tempo, as mensagens SND e REC foram registradas e o valor exibido no dashboard refletiu os eventos processados por cada nó.

11.3 Teste de exclusão mútua

Neste teste, os nós solicitaram acesso à região crítica em momentos diferentes. O objetivo foi verificar se o algoritmo impedia que mais de um nó entrasse simultaneamente na região crítica. O dashboard mostrou alternância entre AGUARDANDO, NA_REGIAO_CRITICA e LIBERADO, as mensagens REQ e REP apareceram nos logs e não foi observado acesso simultâneo de dois nós à região crítica durante a demonstração.

11.4 Teste de eleição de líder

Neste teste, o algoritmo Bully foi observado em dois momentos, na eleição inicial e na recuperação após falha do líder. O líder foi exibido no dashboard, quando o processo do líder foi interrompido, os demais nós identificaram a ausência de heartbeat, uma nova eleição foi iniciada e o maior nó ativo assumiu como novo líder.

11.5 Teste do dashboard

Também foi testada a visualização do dashboard. O objetivo foi verificar se o painel refletia o estado dos nós sem interferir na execução dos algoritmos. A tabela foi atualizada periodicamente, o status online/offline mudou conforme os nós enviavam ou deixavam de enviar mensagens e o painel facilitou a análise visual dos algoritmos, principalmente durante a apresentação em vídeo.

12. Discussão dos resultados

Os resultados mostram que o projeto conseguiu implementar os três algoritmos principais do projeto: relógio lógico de Lamport, exclusão mútua e eleição de líder usando troca real de mensagens.

No caso do relógio de Lamport, a implementação mostra corretamente a atualização do relógio local com base no maior timestamp conhecido, permitindo observar a ordenação lógica de eventos mesmo sem relógio físico global.

No caso da exclusão mútua, a solução implementada com mensagens REQ e REP garante que o acesso à região crítica dependa da permissão dos demais nós ativos. A comparação entre (timestamp, id) permite resolver conflitos quando dois nós solicitam o recurso em momentos próximos.

No caso do Bully, o uso de heartbeats e timeout permite detectar a queda do líder e iniciar uma nova eleição. O comportamento observado condiz com a ideia do algoritmo em que o nó ativo com maior identificador deve se tornar coordenador.

A decisão de usar um dashboard separado foi positiva, porque deixou os testes mais compreensíveis. Em vez de depender apenas dos logs no terminal, foi possível visualizar os estados principais em uma tabela única, o que também ajudou a organizar a visualização.

14. Fontes utilizadas e alterações feitas

Ao final deste relatório, são apresentadas as fontes utilizadas como referência para o desenvolvimento do trabalho. Essas fontes foram consultadas principalmente para compreender os conceitos teóricos dos algoritmos distribuídos implementados e desenvolvidos ao longo da disciplina, como relógios lógicos de Lamport, exclusão mútua distribuída e eleição de líder. No entanto, não foi utilizado código pronto de terceiros de forma direta na implementação do projeto. O código entregue foi desenvolvido com base no entendimento dos algoritmos e nas decisões de implementação adotadas para o ambiente do trabalho, utilizando comunicação por sockets TCP e mensagens em JSON.

15. Comentários sobre a experiência

A realização do trabalho ajudou a entender melhor a diferença entre conhecer um algoritmo distribuído teoricamente e implementá-lo de fato. Na teoria, os algoritmos

costumam ser apresentados com hipóteses bem definidas, como meios confiáveis, nós conhecidos e ausência de certos tipos de falha. Na prática, mesmo em um ambiente controlado, surgem detalhes importantes, como portas, IPs, tempo de espera, concorrência entre threads, envio de mensagens inválidas, processos que podem cair e necessidade de visualizar o estado do sistema.

Outro ponto importante foi perceber que os três algoritmos se complementam. O relógio lógico ajuda a organizar eventos, a exclusão mútua usa timestamps para decidir prioridade e a eleição de líder lida com coordenação quando um nó precisa assumir um papel especial. Assim, o projeto permitiu observar vários conceitos centrais de sistemas distribuídos funcionando em conjunto.

O dashboard também foi uma parte importante do projeto, porque tornou o comportamento distribuído mais visível. Sem ele, seria necessário acompanhar vários terminais ao mesmo tempo, o que dificultaria a análise e a apresentação. Com o painel, ficou mais fácil demonstrar a evolução dos relógios, o líder atual e o estado da região crítica.

16. Conclusão

O projeto atendeu aos objetivos propostos no enunciado do trabalho, implementando três algoritmos distribuídos com comunicação real por mensagens, incluindo o relógio lógico de Lamport, exclusão mútua baseada em Ricart-Agrawala e eleição de líder com Bully.

A solução foi executada em um ambiente distribuído na Google Cloud, com quatro nós de trabalho e um dashboard separado para visualização. A comunicação foi feita por sockets TCP com mensagens JSON e os algoritmos foram implementados em Python 3. Os testes mostraram o funcionamento esperado dos principais mecanismos, incluindo atualização de relógios lógicos, controle de entrada na região crítica e eleição de novo líder após falha.

Como resultado, o trabalho permitiu aplicar na prática conceitos fundamentais de sistemas distribuídos, incluindo troca de mensagens, concorrência, sincronização lógica, coordenação entre processos e tolerância básica a falhas.

17. Referências

Tanenbaum, Andrew S, Van Steen, Maarten. Distributed systems: principles and paradigms. Pearson/Prentice Hall. 4th Ed. (<https://www.distributed-systems.net/index.php/books/ds4/>)

Coulouris, G, Dollimore, J., Kindberg, T. Distributed Systems: concepts and design. 4th Ed.

Kshemkalyani, Ajay D., Singhal, Mukesh. Distributed computing: principles, algorithms, and systems. Cambridge University Press.

Corrêa, R. Dutra, I., Fiallos, M., Gomes, F. (Eds.). Models for Parallel and Distributed Computation: Theory, Algorithmic Techniques and Applications. Kluwer Academic Publishers.

Barbosa, V.C. Massively Parallel Models of Computation. Ellis Horwood Limited.

Sinnen, Oliver. Task Scheduling for Parallel Systems. Wiley.